SVP Program Report

Calvin Dantis zmzf47

1 Reasoning

In lower dimensions, enumeration is faster than lattice reduction but less scalable. Schnorr-Euchner (SE) enumeration[1] is an exact SVP algorithm but it is inefficient with low orthogonality bases and impractical at higher dimensions. Lattice reduction is often used as a preprocessing step for exact SVP algorithms. It can increase running time in lower dimensions however, making it more suitable for higher dimensions instead. Enumeration is preferable over alternatives like sieving or Voronoi cell reduction due to its polynomial space complexity, despite its higher time complexity. However, this speed difference is negligible in lower dimensions.

2 Matrix

The matrix class represents an n x n matrix using a double pointer array with n double arrays of size n. During initialization, the matrix is filled with values that can be modified through a referencing class function. Arrays are preferred for efficiency, removing dynamic memory allocation overhead. The destructor ensures safe destruction, preventing memory leaks.

3 Parser

The matrix dimension is calculated by square rooting the number of arguments as each basis value corresponds to a unique argument in a square input basis; this then initialises a square matrix which is sequentially filled by parsing each argument with modular arithmetic. Checks are implemented to catch input errors and prevent solver issues.

4 SVP Solver

Matrix information containers, including Gram-Schmidt norms and coefficients, are defined. To minimise space, the coefficient array stores only $\frac{n(n-1)}{2}$ elements instead of the naive n^2, and an index function locates a column's coefficients. The initialisation function, based on the stage preparation of FP-LLL[1], calculates only the necessary Gram-Schmidt information to minimise redundant computations. Constants are also used whenever possible to better utilise the compiler's optimisation.

The enumeration algorithm closely follows Masaya Yasuda's optimised version of SE enumeration [2], minimising arrays and removing redundancies. Data referencing optimisations greatly improve performance: column and coefficient indexes are referenced and stored before they are iterated on for quicker access. The enumeration radius is set as an upper bound of the shortest norm using Hermite's constant[3]: $\lambda_1(L) \leq \sqrt{\gamma_n} vol(L)^{\frac{1}{n}}$, significantly reducing the number of enumerations required. For dimensions 2-8, Hermite constants are known and preprocessed, however other dimensions use an estimate of $\lambda_1(L) \leq (\frac{2}{\pi})^{\frac{1}{2}} (\Gamma(2 + \frac{n}{2}) vol(L))^{\frac{1}{n}}$[4]. The bound requires the volume of the lattice which is calculated by producing the Gram-Schmidt norms[2]. As the square root is computationally expensive, the square norms are produced then rooted at once during the bound calculation.

$$vol(L)^{\frac{1}{n}} = det(B)^{\frac{1}{n}} = \left(\prod_{i=1}^{n} \|b_i^*\|\right)^{\frac{1}{n}} = \left(\prod_{i=1}^{n} \|b_i^*\|^2\right)^{\frac{1}{2n}}$$

5 Complexity

Scalability is the largest drawback. The time complexity of parsing is $\Theta(n^2)$ due to the constant n^2 arguments. While enumeration's super-exponential[2] running time has minimal impact at low dimensions, its effects become noticeable at >50 dimensions, taking several seconds to execute.

The program's space complexity is $\Theta(n^2)$ due to matrix storage, unavoidable without sacrificing correctness. The solver's space complexity is also $\Theta(n^2)$ as the container for Gram-Schmidt coefficients grows quadratically with dimension.

SE enumeration is more scalable than pure lattice reduction, always finding the exact shortest norm. However, when combined with an exact SVP algorithm, lattice reduction becomes more scalable, with lower average complexity and outperforming enumeration at higher dimensions.

6 Performance

The program's performance is evaluated in several dimensions, measuring execution time and storage usage. Tests use uniformly randomly generated bases with 2 decimal point values between (-1000, 1000). and the canonical standard basis, FP-LLL is also implemented and tested on random bases. The average execution time of 100,000 runs is recorded. The file writing process is not measured.

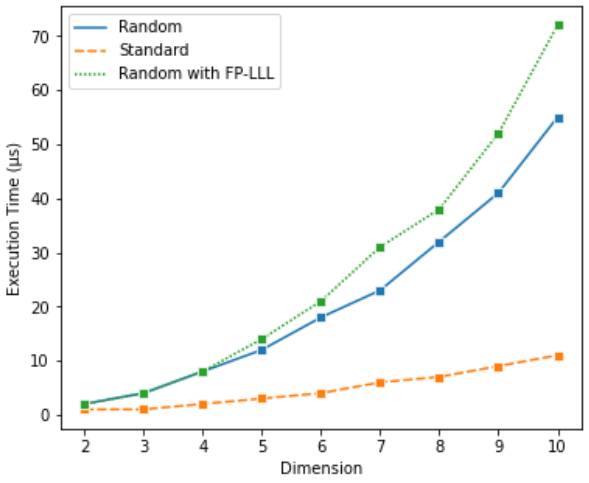

Figure 1 Execution times at different dimensions

At low dimensions, the execution time of the standard basis is nearly linear, as the algorithm simply traverses each vector without computation. The complexity becomes apparent as the dimension increases. However, pure enumeration consistently has lower or equal execution times compared to running it with lattice reduction, and this gap widens with increasing dimension in this low-dimensional context.

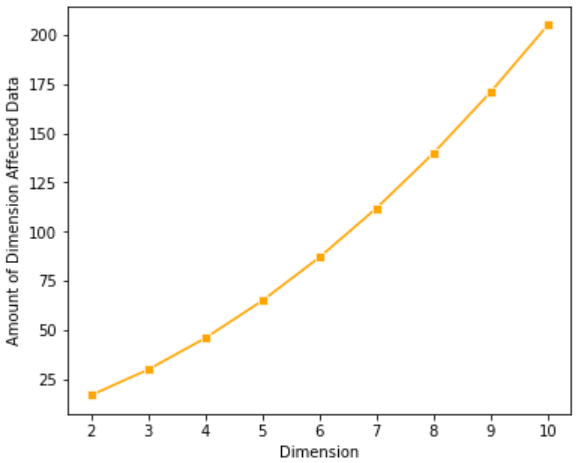

Figure 2 Amount of data stored in containers affected by input dimension

Several containers are dimension-dependent, modelled by an equation to determine a general trend for storage utilisation. The parser utilises n^2 space and the solver requires $6n + \frac{n(n-1)}{2}$ space. The total dimension-dependent data can be represented by the polynomial equation:

$$n(n + \frac{(n-1)}{2} + 6)$$

7 References

[1] C. P. Schnorr and M. Euchner, "Lattice basis reduction: Improved practical algorithms and solving subset sum problems," *Mathematical Programming*, vol. 66, no. 1–3, pp. 181–199, 1994. doi:10.1007/bf01581144

[2] M. Yasuda, "A survey of solving SVP algorithms and recent strategies for solving the SVP Challenge," *International Symposium on Mathematics, Quantum Theory, and Cryptography*, pp. 189–207, 2020. doi:10.1007/978-981-15-5191-8_15

[3] Ch. Hermite, "Extraits de lettres de M. Ch. Hermite à M. Jacobi sur différents objects de la théorie des nombres.," *Crelle's Journal*, vol. 1850, no. 40, pp. 261–278, Jul. 1850, doi: https://doi.org/10.1515/crll.1850.40.261.

[4] Y. Kitaoka, *Arithmetic of quadratic forms*. Cambridge: Univ. Press, 2003.